6.14 Longest (Lab 6.2)

LAB ACTIVITY | 6.14.1: Longest (Lab 6.2) | Full screen | 0 / 30

Computers can read through long lists of objects, filtering or collecting aggregate statistics as they go. The *accumulator pattern* is a common algorithm design for doing this type of work. In this exercise, you will read through a list of words (strings) looking for the longest. In this exercise, we ask you to design the algorithm yourself.

Download this template and rename it `longest.py`:

- template.py

Design and implement a function, `find_longest`, that receives a list of strings and returns the longest string in the list. If several strings are tied for the longest, your function may return any one of them. If the list is empty, return an empty string.

Some test cases:

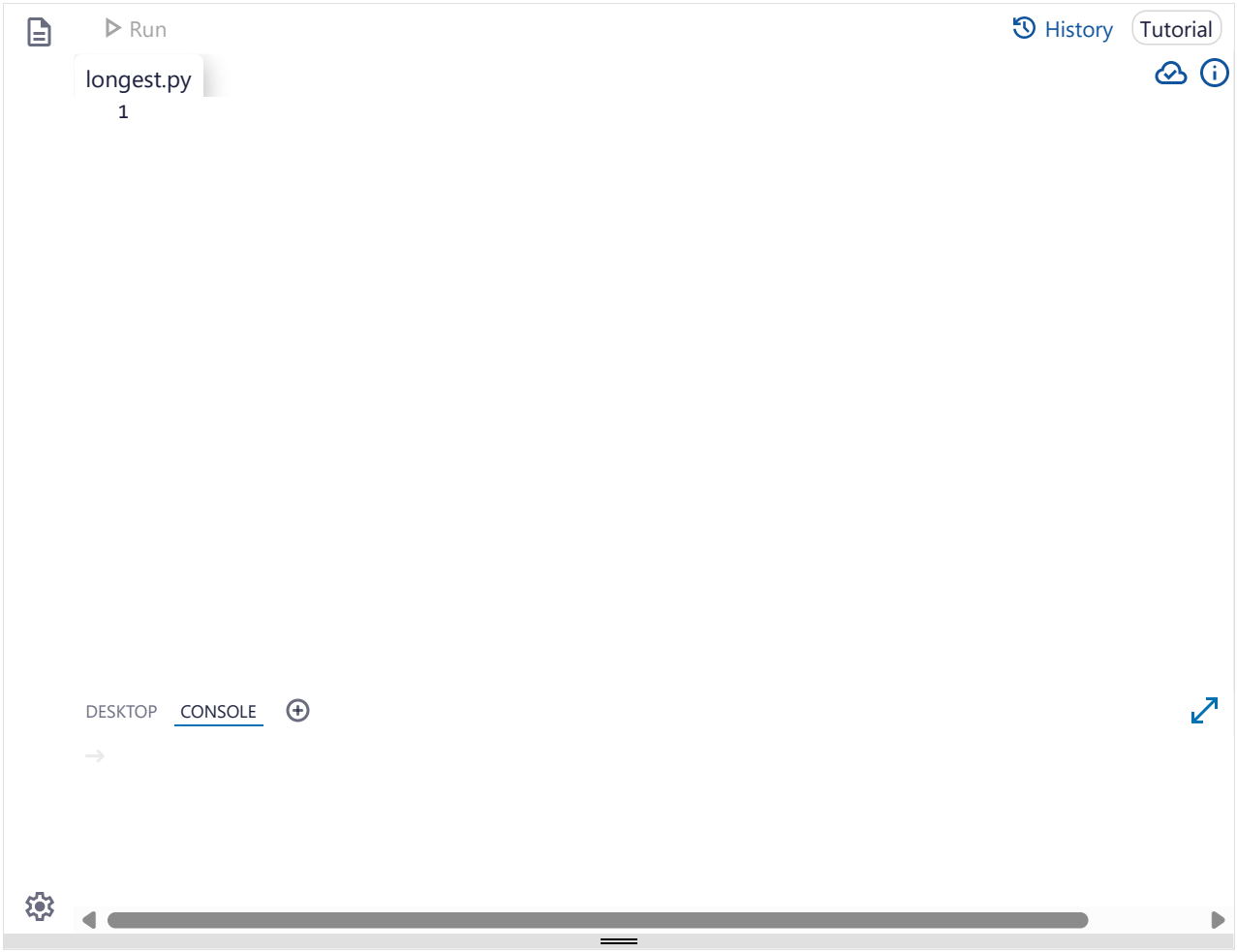
List argument value	Return value
`['for', 'he', 'is', 'a', 'very', 'good', 'fellow']`	`'fellow'`
`['one', 'two']`	either `'one'` or `'two'`
`[]`	`''`

Remember that algorithms implementing an accumulator pattern using a function have the following general structure:

Given: Receive data from the calling program.
Initialize the accumulator appropriately.
Repeat for each element in a list
    Update the accumulator appropriately.
Return the accumulator value.

In this exercise, the function is given the list of strings to search and the accumulator needs to keep track of the longest word found so far in the loop through the list elements. To help make this clear, we suggest that you name your accumulator `longest_word_so_far`. You'll want to initialize this accumulator to some really small word that couldn't possibly be longer than any of the words in the list.

Open new tab | Dock



Submit for grading

Coding trail of your work What is this?

```
History of your effort will appear here once you begin working on this
zyLab.
```

Latest submission

No submissions yet

Template, Model Solution, and Test Bench Show

Lab statistics Show

Student behavior insights (beta) Show

Similarity detection Show

Student activity Show